

DDQN-TS: A novel bi-objective intelligent scheduling algorithm in the cloud environment

Zhao Tong^{*}, Feng Ye, Bilan Liu, Jinhui Cai, Jing Mei

College of Information Science and Engineering, Hunan Normal University, Changsha 410012, China

ARTICLE INFO

Article history:

Received 28 October 2020

Revised 13 April 2021

Accepted 21 May 2021

Available online 25 May 2021

Communicated by Zidong Wang

Keywords:

Cloud computing

Deep reinforcement learning

Double deep Q-network

Quality of service

Task scheduling

ABSTRACT

Task scheduling has always been one of the crucial problem in cloud computing. With the transition of task types from static batch processing to dynamic stream processing, the dynamic online task scheduling problem has attracted widespread attention. At this stage, explore an effective task scheduling method to implement high quality of service (QoS) requests with limited resources is a considerable challenge. This paper proposes a novel scheduling algorithm called double deep Q-network task scheduling (DDQN-TS), which uses the adaptive learning ability of double deep Q-network (DDQN) to explore the optimal task scheduling strategy. Experiments conducted using the Random, Google, and Alibaba benchmarks to compare several classic algorithms show that the proposed DDQN-TS can guarantee a high task completion rate and efficiently reduce the task average response time.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Cloud computing involves the integration and development of technologies, such as distributed computing, utility computing, virtualization technology, Web services, and grid computing. The goal is for users to maximize the use of virtual resource pools and processes for large-scale calculation problems. Compared to traditional computing methods, cloud computing emphasizes the way services are delivered on-demand. Cloud computing is characterized by scalability, high reliability, measurability, poolable service resources, and diversified service resources [1]. The various types of cloud computing have different divisions according to different perspectives. From the perspective of deployment, cloud computing is divided into the public cloud, private cloud, community cloud, and hybrid cloud [2]. From the perspective of services, cloud computing can be divided into three types: infrastructure as a service (IaaS), software as a service (SaaS), and platform as a service (PaaS) [3].

Due to the variety of requests from users, there is diversity of resources, different types of servers and various quality of service (QoS) target constraints [4]. Cloud computing provides different levels and types of services and processes many data and tasks continuously. Under such circumstances, determining how to reasonably allocate tasks under limited resource conditions is an incredibly important decision, both to ensure the user's request

and to satisfy the requirements of the cloud service provider [5,6]. Ensuring a high throughput rate without affecting the system's transmission rate and the load balancing of the entire system is an important problem in technology.

Virtualization is one of the essential features of cloud computing, which can virtualize hardware resources, software resources, and operating platforms. Virtualization technology can support multiple applications on the same computer node to share various available physical resources and improve the computing nodes' utilization to achieve efficient calculation. However, due to the dynamic and volatile nature of the cloud computing environment and the uncertainty of tasks that users continuously submit [7,8], efficient task scheduling under limited resource conditions remains a considerable challenge. Compared to traditional distributed computing and grid computing [9,10], the cloud environment and cloud characteristics make solving task scheduling problems for cloud-based applications challenging. First, due to the different needs of users, IaaS vendors usually provide only one type of virtual machine but various types of virtual machine instances for users to select the appropriate virtual machine according to their needs [11]. Second, due to the large number of virtual machine instances, there will be large differences in workloads and changes in real time [12]. Finally, cloud-based applications are capable of successfully performing a variety of tasks while ensuring various user QoS requirements [13].

There are many indicators for evaluating performance in the cloud environment, such as the makespan, load balancing, eco-

^{*} Corresponding author.

E-mail address: tongzhao@hunnu.edu.cn (Z. Tong).

nomic principles, and average response time [14–18]. However, for different types of task requests, the metrics differ. Tasks submitted by general users are mainly divided into two categories: offline and online tasks. For online tasks, scheduling begins when all tasks have been submitted, so the broker knows the task's arrival time, the various resources required, and other attributes. Therefore, the main considerations for this type of task are to minimize the total cost, balance the load, and reduce the total completion time. In this paper, we aim to optimize the scheduling of online tasks, which is difficult to perform in a dynamic and uncertain cloud environment [19]. Because throughout the scheduling process, the broker does not know how many tasks will arrive and how many resources each task will request, for real-time scheduling, whenever a task arrives, the task is assigned to the most appropriate virtual machine for execution. Therefore, for this kind of task, the main consideration is to reduce the average response time of the task to guarantee a high QoS. The new method we are investigating must learn to make intelligent task scheduling decisions so that users' tasks can be successfully executed and the users can achieve high satisfaction with limited resources.

To achieve the optimization objectives, we propose an intelligent task scheduling framework based on DDQN algorithm. When Mnih et al. [20] first proposed the deep Q-network (DQN) algorithm, it was mainly used in game scenarios [21]. In recent years, with the application of DQN in artificial intelligence (AI) control problems, the DQN algorithm has been widely applied to task scheduling problems in cloud environments [22,23]. The DQN algorithm is a combination of reinforcement learning (RL) and a deep neural network (DNN). It has the ability of RL to learn good strategies to solve sequential problems and the ability of the DNN to fit high-dimensional states. Therefore, the DQN algorithm can be effectively applied to complex task scheduling problems. However, neither RL nor DQN algorithms consider overestimation of action values during the scheduling process. In response to this deficiency, Van Hasselt et al. [24] proposed the DDQN algorithm. We apply the DDQN algorithm in our work with a slight improvement. We have evaluated our proposed method under three different benchmarks: Random, Google, and Alibaba. Our proposed task scheduling method has clear advantages compared to three classic algorithms in terms of the average response time and task completion rate. Moreover, compared to the DQN algorithm, the proposed method effectively reduces the overestimation of the action values and accelerates the network's convergence.

The main contributions of this paper are as follows:

- With the extensive application of deep reinforcement learning (DRL) in task scheduling problems, we propose an intelligent task scheduling framework using the DDQN algorithm in the cloud computing environment.
- We base our approach on the characteristics of real-time tasks submitted by users and propose a combination of offline training to perform intelligent scheduling and an online decision-making scheduling strategy.
- We use the DDQN algorithm to effectively reduce the overestimation of the action values during the scheduling process and accelerate the convergence of the network.
- We use three different benchmark datasets and conduct comparative experiments with several different algorithms. The results indicate that our proposed method has distinct advantages in terms of both the average response time and task completion rate.

The rest of this paper is organized as follows. Section 2 reviews related work on scheduling algorithms. Section 3 describes the system model, task model, and scheduling problems. Section 4 explains Q-learning, the DRL method, and the main content and

design principle of DDQN-TS. Section 5 compares the performance of our proposed algorithm with that of several other algorithms and analyzes the experimental results. Finally, Section 6 summarizes this paper and provides potential directions for future work.

2. Related work

Task scheduling [25,26] is an indispensable but challenging problem in the field of parallel distributed systems. In the cloud computing environment, task scheduling is a key factor affecting the cloud computing performance. Similar to the predecessor of grid computing, various task scheduling approaches have been proposed [15–18]. In particular, with the breakthroughs in deep learning (DL) and RL in the field of AI, various RL and DRL approaches have been applied to task scheduling [23,27,28].

Task scheduling aims to solve the problem of how to implement a reasonable allocation of resources via effective scheduling strategy and reasonable optimization approaches [29]. However, effective scheduling methods are often based on theoretical research that employs statistical theories, such as control theory [30–32], game theory [33,34] and queuing theory [35,36] to build models of the underlying mathematical systems. In terms of queuing theory, the simplest cloud computing model is the queuing model in the form of $M/M/1$. Due to the dynamics of network traffic and the VM environment running in the model, the simplest cloud computing model ignores many important details. Therefore, it is usually used only as a benchmark and starting point for other cloud computing queuing models. In [35], the authors assume that the time interval between task arrival and service time is exponentially distributed with a fixed buffer size $m + r$, so they modeled the cloud center as an $M/M/m/m + r$ queuing model, enabling the agent to easily obtain response times for task requests. Aldossary et al. [30] proposed a control theory solution for the dynamic capacity supply problem that minimizes the total energy cost while meeting the task scheduling delay performance goal. Specifically, they fully exploited the advantages of control theory to model the problem as a constrained discrete-time optimal control problem and used model predictive control (MPC) to find the optimal control strategy. As cloud-based services become more numerous and dynamic, reasonable scheduling of resources becomes increasingly more challenging. To achieve efficient resource allocation under QoS constraints, Liu et al. [34] proposed an approximate practical solution based on game theory and achieved good results. Various task scheduling methods based on statistical theory can achieve efficient task scheduling. Therefore, by combining the scenarios considered in this paper with actual problems to be solved, we propose a task scheduling method based on queuing theory.

In the field of AI, machine learning is an active learning method that enables programs to automatically learn patterns or models of an underlying system. RL is an important machine learning algorithm that learns good strategies for solving sequential decision problems by optimizing the accumulated future reward signals. The Q-learning algorithm is one of the most popular RL algorithms. Therefore, many task scheduling methods based on Q-learning have been proposed. Tong et al. [28] proposed a novel task scheduling algorithm, QL-HEFT, based on Q-learning and the heterogeneous earliest finish time (HEFT) algorithm. The scheduling method can achieve efficient task scheduling and reduce the makespan. Balla et al. [36] proposed a novel task scheduling scheme based on Q-learning and queuing theory to optimize task scheduling under a dynamically changing cloud environment and developed an adaptive action-selection method to control the action selection (suitable virtual machine selection) dynamically. The final results demonstrate the effectiveness of the scheduling method.

Although RL can be used to effectively solve some task scheduling problems, limitations remain. For example, due to the restriction on the size of the Q-table, Q-learning can be effectively applied only to scenarios with discrete events and low-dimensional states. To avoid the drawback of Q-learning and solve complex task scheduling problems in the cloud environment, DRL methods have been effectively applied. Demirel et al. [37] proposed a control-aware scheduling (DEEPCAS) algorithm based on DRL and control theory to solve the task scheduling problem. The experimental results show that the proposed DEEPCAS algorithm outperforms the periodic scheduling method. Cheng et al. [38] proposed a novel DRL-Cloud system based on DRL that has two stages of resource provisioning and task scheduling to reduce the energy costs of cloud service providers with large-scale data centers and a large number of user requests. The scheduling method achieves low total energy consumption and low task execution time while ensuring a low task rejection rate. Zhou et al. [39] proposed two novel adaptive energy-aware algorithms to maximize the energy efficiency and minimize the service level agreement (SLA) violation rate in Cloud data centers. To study the efficacy of the proposed approaches, they performed extensive experimental analyses using real-world workloads. The experimental results show that the proposed approaches are effective. Wei et al. [40] proposed an intelligent QoS-aware job scheduling framework for application providers based on the DQN algorithm to achieve efficient scheduling under QoS constraints. The experimental results show that the job scheduling method effectively reduces the average response time of jobs while ensuring high QoS. Nevertheless, while the original DRL algorithm and DQN algorithm can solve the inevitable dimensional disaster of Q-learning encountered in large-scale problems, regardless of whether the original DRL algorithm or a novel DQN algorithm is used, unrealistically high action values are sometimes produced. This occurs because these methods include a maximization step that exceeds the estimated action value, so they tend to overestimate rather than underestimate such values. Therefore, to effectively alleviate the above problems, this paper designs an online task scheduling framework based on the DDQN algorithm.

To the best of the authors' knowledge, the DDQN algorithm is currently used mainly in Atari game scenarios, so few works that use the DDQN algorithm to solve task scheduling problems in the cloud computing environment exist. The following section formulates the problems considered in this article.

3. Task scheduling process and problem description

In this section, we first define the task scheduling process in the cloud environment. Then, we describe the task types submitted by users and the cloud resources of the data center. Finally, we present a formal description of scheduling problems.

3.1. Online task scheduling process in the cloud environment

In the cloud computing environment, task scheduling is an NP-hard problem [41], where the goal is to find an approximate optimal solution in the solution domain. Specifically, the goal is to allocate appropriate computing and storage resources to tasks to improve the performance of task scheduling. When a user submits a task, the tasks are allocated to virtual machines (VMs) based on the scheduling strategy. Therefore, a reasonable choice of scheduling strategy is important to optimize the task completion rate and average response time. Fig. 1 shows a model of online task scheduling in a cloud computing scenario. First, the user submits a task to the data center in real time. Then, the data center agent allocates VMs to the task according to a scheduling policy and different

QoS requests (e.g., availability and accessibility of computing nodes, throughput, network bandwidth, memory size, CPU computing power, deadline, load balancing, and economic principles). Once the task is assigned, it enters the task waiting queue of the corresponding VM instance and waits for execution. When no task waiting queues have a task waiting to be executed, the task scheduling is completed. Notably, in the task scheduling model, since tasks are submitted by users in real time, the total number of tasks and the quantity of requested resources are unknown until all tasks arrive at the data center. Moreover, we assume that the tasks submitted by a user are non-preemptive, and that VMs are heterogeneous and scalable, but the network bandwidth is the same.

3.2. Task type and cloud resource description

Tasks can be divided into dependent tasks and independent tasks, depending on whether there are dependencies between tasks. A priority does not exist between independent tasks. Priority and constraint relationships exist between dependent tasks, and subtasks must wait until all their parent tasks have been completed before they can begin execution. According to the real-time nature of task submission, tasks can be divided into offline tasks and online tasks. For offline tasks, before the scheduling begins, the broker already knows the total number of tasks, including the arrival time of each task, the size of each task, the resources requested by each task, and other attributes. For online tasks, before the scheduling begins, the broker does not know how many tasks will be submitted until they arrive, and the broker has no knowledge of the size of each task, the number of resources required, and other attributes. In our model, we adapt a real-time scheduling method: each time a task arrives, if the available resources are sufficient, the task is executed. Therefore, in this paper, we consider independent online tasks. Based on the above assumptions, each task can be expressed according to Eq. (1).

$$Task_i = \{T_i^{id}, T_i^{arrival}, T_i^{size}, mem_i, cpu_i, QoS_i\} \quad (1)$$

where $Task_i$ represents task i , T_i^{id} represents the id of task i , $T_i^{arrival}$ represents the arrival time of task i , T_i^{size} represents the data size of task i , mem_i represents the memory resources required to execute task i , cpu_i represents the CPU resources required to execute task i , and QoS_i represents the QoS requirements of task i (that is, the minimum requirements for task i to successfully execute).

Given that users submit tasks that differ in size, there are some tasks with a large data size and others with a small data size. Therefore, cloud service providers lease VMs of various execution speeds to meet the needs of various types of tasks. In our model, we assume there is a limited number of small-scale VM instances in the resource pool and that each VM instance can be represented by Eq. (2).

$$VM_j = \{VM_j^{id}, VM_j^{type}, VM_j^{speed}\} \quad (2)$$

where VM_j represents VM j , VM_j^{id} represents the id of VM j , VM_j^{type} represents the execution speed of VM j (e.g., high speed, medium speed, and low speed), and VM_j^{speed} represents the execution speed of VM j .

The notation used in this paper is summarized in Table 1.

3.3. Task scheduling problem

In general, the task scheduling problem involves task assignment and execution under certain QoS constraints. As QoS requirements, we consider two performance metrics that affect the

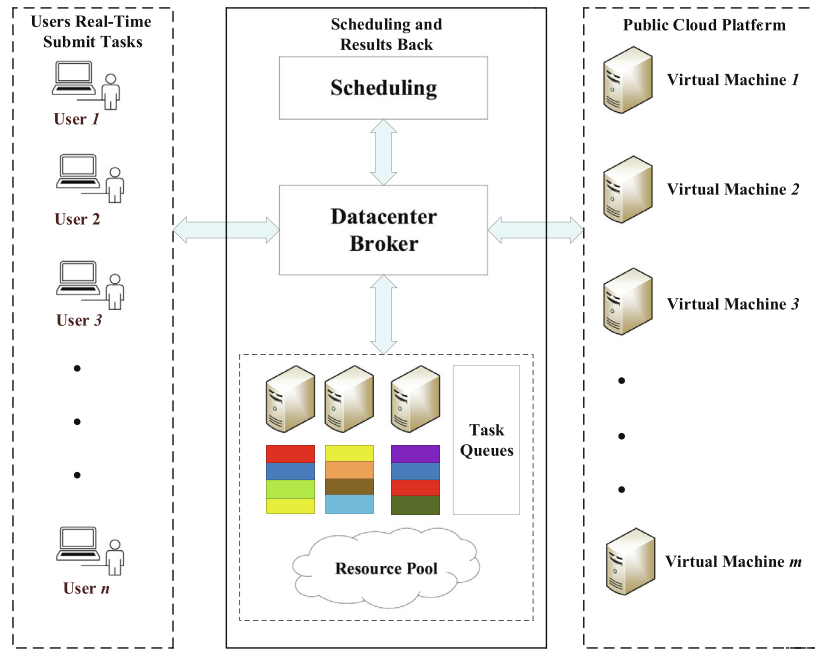


Fig. 1. Process of online task scheduling in cloud computing.

Table 1

Notations.

Name	Description
H	The total number of tasks that can be successfully executed
m	The total number of VMs
$Task_i$	Task i
$Task_i^r$	The next subtask of task i
T_i^{id}	The id of task i
$T_i^{arrival}$	The arrival time of task i
T_i^{size}	The data size of task i
QoS_i	The minimum requirement for task i to execute successfully
VM_j	VM j
VM_j^{id}	The id of VM j
VM_j^{type}	The execution speed level of VM j
VM_j^{speed}	The execution speed of VM j
VM_j^{speed}	The execution speed of the slowest VM
T_{ij}^{rt}	The response time that task i is assigned to execute on VM j
T_{ij}^{exe}	The execution time that task i is assigned to execute on VM j
T_{ij}^{wait}	The time that task i waits for execution in the waiting queue of VM j
L_j^{queue}	The number of tasks waiting to be executed in the task waiting queue of VM j
VM_{ij}^{idle}	The earliest idle time of VM j before execution of task i

efficiency of task scheduling. One is to guarantee a maximum completion rate of the task (that is, the successful execution rate of the task). The other is to minimize the average response time of all tasks while ensuring successful execution. Since our model uses an online decision-making approach, whenever a task arrives, it will be assigned immediately. Furthermore, the agent estimates whether the task defaults according to the SLA. Once the task defaults, the task will not enter the task waiting queue and will not be executed again. If the task does not default, it will enter the task waiting queue for the VM instance to which it is assigned. We assume that the task waiting queue is sufficiently long to store all unexecuted tasks temporarily. Tasks in the task waiting queue are scheduled on a first-come, first-served (FCFS) basis. Each VM

instance can execute only one task at any time. That is, preemption is not allowed, which means that no task can be interrupted during execution. If a task waiting queue is empty, once a new task arrives in the queue, it can be executed immediately. Clearly, if there are tasks waiting to execute in a waiting queue, the newly arrived task will wait for all tasks to be executed before it can be executed.

In this paper, we define the task response time T_{ij}^{rt} as the sum of the time that $Task_i$ waits to execute and the execution time. This value can be expressed as Eq. (3).

$$T_{ij}^{rt} = T_{ij}^{exe} + T_{ij}^{wait} \quad (3)$$

where T_{ij}^{exe} represents the execution time assigned to VM_j by $Task_i$ and T_{ij}^{wait} represents the time that $Task_i$ waits for execution in the waiting queue of VM_j . The average response time of the task is the ratio of the sum of all task response times to the total number of tasks.

The execution time assigned to VM_j by $Task_i$ can be expressed as Eq. (4).

$$T_{ij}^{exe} = \frac{T_i^{size}}{VM_j^{speed}} \quad (4)$$

Suppose L_j^{queue} is the number of tasks waiting to be executed in the task waiting queue of VM_j . Then, the time that $Task_i$ assigned to VM_j waits to be executed can be expressed as Eq. (5).

$$T_{ij}^{wait} = \begin{cases} 0, & \text{if } L_j^{queue} = 0 \\ VM_{ij}^{idle} - T_i^{arrival}, & \text{else} \end{cases} \quad (5)$$

where VM_{ij}^{idle} represents the earliest idle time of VM_j before the execution of $Task_i$. Suppose $Task_i^r$ is the last task dispatched to VM_j before $Task_i$. Then, the earliest idle time of VM_j before the execution of $Task_i$ can be expressed as Eq. (6).

$$VM_{ij}^{idle} = \begin{cases} VM_{ij}^{idle} + T_{ij}^{exe}, & \text{if } VM_{ij}^{idle} > T_{i^r}^{arrival} \\ T_{i^r}^{arrival} + T_{ij}^{exe}, & \text{else} \end{cases} \quad (6)$$

where VM_{ij}^{idle} represents the earliest idle time of VM_j before the execution of $Task_i$. T_{ij}^{exe} represents the execution time of $Task_i$ on virtual machine VM_j , and $T_i^{arrival}$ represents the arrival time of $Task_i$.

Whether the task is successfully executed is based on the SLA. In general, the SLA is defined by users according to their requirements. In our research, the SLA is defined as in Eq. (7).

$$VM_{ij}^{idle} + T_{ij}^{exe} \leq T_i^{arrival} + (T_i^{size} / VM_i^{speed}) \quad (7)$$

where VM_i^{speed} represents the execution speed of the VM_i that is executing the slowest, and if the task is successfully executed, then the above inequality must be satisfied. Therefore, whether the task is successfully executed can be expressed by Eq. (8).

$$success_{ij} = \begin{cases} 1, & \text{if } VM_{ij}^{idle} + T_{ij}^{exe} \leq T_i^{arrival} + (T_i^{size} / VM_i^{speed}) \\ 0, & \text{else} \end{cases} \quad (8)$$

where $success_{ij} = 1$ means that $Task_i$ can be successfully executed on VM_j and $success_{ij} = 0$ means that $Task_i$ cannot be successfully executed on VM_j . That is, considering the online intelligent decision scheduling of $task_i$ to VM_j , if the sum of the earliest idle time of VM_j and the execution time of $Task_i$ on VM_j is less than or equal to the sum of the arrival time of $Task_i$ and the execution time of $Task_i$ assigned to the VM_j that is executing the slowest, then the task can be successfully executed; otherwise, it will not be executed successfully.

Assuming that the number of successfully executed tasks is H and the response time of each task is T_i^{rt} , the task completion rate (TCR) and average response time (ART) of the entire task can be expressed by Eqs. (9) and (10).

$$TCR = \frac{H}{N} \quad (9)$$

$$ART = \frac{\sum_{i=1}^H T_i^{rt}}{H} \quad (10)$$

Clearly, when solving the response time of a task, we do not consider tasks that cannot be successfully executed because they do not respond. In other words, if the QoS request of a task is not satisfied, then the task will not be rescheduled.

In this paper, our proposed task scheduling method is designed to ensure a low average response time and a high task completion rate. Therefore, the optimization problem for maximizing the task completion rate and minimizing the task average response time can be modeled as follows:

$$\begin{aligned} \max \quad & TCR = \frac{H}{N} \\ \min \quad & ART = \sum_{i=1}^H T_i^{rt} / H \\ \text{s.t.} \quad & C1 : mem_i \leq M, \forall i \\ & C2 : cpu_i \leq C, \forall i \\ & C3 : VM_{ij}^{idle} + T_{ij}^{exe} \leq T_i^{arrival} + (T_i^{size} / VM_i^{speed}), \forall i, j \end{aligned} \quad (11)$$

where mem_i and cpu_i are the memory and CPU resources of $Task_i$ required to execute the request, and M and C represent the memory and CPU resources of the VM, respectively. Because the task execution is waiting for the queue, we consider it to be non-preemptive, and each computing node executes a single task. Therefore, the computing resources requested by our default task are always smaller than the resources available at the computing node.

4. Algorithm design

Because our proposed task scheduling algorithm is based on RL and DRL, this section first describes the Q-learning algorithm, which is one of the most popular RL algorithms. Then, we introduce the DRL method. Finally, we present the details of our proposed task scheduling algorithm.

4.1. Q-Learning

Q-learning is an RL [42] framework based on MDPs [43] that can make optimal action decisions in a limited MDP environment. The working mechanism is continuous trial-and-error in a dynamic environment without prior knowledge. By learning a large number of different actions, obtaining reasonable feedback and reaching the final state, the final converged Q-table is obtained. In a given state, the agent queries the Q-table for the maximum Q-value of the current state and performs the corresponding action, thereby obtaining an optimal solution. The Q-learning model includes an agent, a finite state set S , an available action set A for the agent, and the reward r after an action is executed and all learned information is stored in the Q-table. In each iteration of the Q-learning algorithm, the agent first observes the state s_t of the current environment and then selects an action a_t based on a particular policy. After executing this action, the agent will obtain a reward value r from the environment. Furthermore, the system will transition from the current state to the next state s_{t+1} . At the beginning of the next iteration, the Q-value is updated according to Eq. (12).

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha [r + \gamma \times \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (12)$$

where r represents the reward value and α is the learning rate, which ranges from 0 to 1. The discount factor γ ranges from 0 to 1: the closer the value of γ is to 0, the more attention the agent pays to the current reward. Conversely, the closer the value is to 1, the more the agent pays attention to the future reward. $\max_a (Q(s_{t+1}, a))$ represents the maximum Q-value obtained when executing different actions a in the current state s_t .

4.2. Deep reinforcement learning

Although Q-learning can effectively solve some task scheduling problems, the algorithm has many limitations. The traditional Q-learning algorithm describes the value function in tabular form; that is, it stores all Q-values in a table called the Q-table. However, since the scale of the Q-table cannot be infinite and real problems are multidimensional, the high-dimensional state and action space will lead to dimensional disasters, which in turn seriously affect the convergence speed of Q-learning. Therefore, Q-learning is incapable of solving complex control problems with high-dimensional states and actions. To overcome this drawback, in DRL algorithms, a function approximator is used to estimate the action value function, $Q(s, a, \theta) \approx Q^*(s, a)$. DRL was proposed as a combination of RL and deep neural networks (DNN) [44]. A neural network function approximator with weights θ is often referred to as a Q-network. A Q-network can be trained by minimizing a sequence of loss functions $Loss(\theta_t)$ that changes in each iteration, and the loss function can be expressed as Eq. (13).

$$Loss(\theta_t) = E[(y_t - Q(s_t, a; \theta_t))^2] \quad (13)$$

where y_t is the target Q-value calculated by the Q-network and $Q(s_t, a; \theta_t)$ is the current Q-value. The target Q-value y_t in the update formula can be expressed as Eq. (14).

$$y_t = r + \gamma \max_{a'} Q(s_{t+1}, a'; \theta_{t-1}) \quad (14)$$

where θ_{t-1} is a parameter of the Q-network. The essence of DRL is to reduce the Loss value through continuous optimization, and finally make the Loss tend to be stable and close to 0. The Loss is optimized by error back-propagation technology. With the continuous training, the value of Loss is constantly changing, and the parameters of Q-network are constantly adjusted.

4.3. DDQN-TS algorithm

At present, although related algorithms in RL and DRL are widely used in cloud environments to solve task scheduling and resource allocation problems, many algorithms sometimes learn unrealistically high action values, such as: Q-learning, DQN etc. This occurs because these algorithms include a maximization step that exceeds the estimated action Q-value, so such algorithms tend to overestimate rather than underestimate values. To alleviate the overestimation of the action Q-value and accelerate the convergence of the network, Liang et al. [27] proposed the DDQN algorithm. The DDQN algorithm has two identical DNN, one named Q-network, the other named target Q-network. Q-network is used to calculate the current Q-value and update the network parameters; target Q-network is used to calculate the target Q-value, and the target Q-network does not need to update the network parameters. Every specific time step, Q-network copies its network parameters to the target Q-network. The target Q-value y_t update formula can be expressed as Eq. (15).

$$y_t = r + \gamma \times Q(s_{t+1}, \arg\max_a Q(s_{t+1}, a, \theta_t); \theta_t^-) \quad (15)$$

where θ_t, θ_t^- are the parameters of Q-network and target Q-network respectively. $\arg\max_a Q(s_{t+1}, a; \theta_t)$ represents the action a corresponding to the maximum Q-value obtain when the Agent executes all the decisions at time t , the selection of action a is completed in the Q-network. After obtaining the maximum action a , the action is used to calculate the target Q-value in the target Q-network. This process can be understood as two steps: the selection of decoupled target Q-value action and the calculation of target Q-value. The DDQN algorithm is through these two steps to eliminate the overestimation of action Q-value, so as to speed up the convergence speed of the algorithm. The architecture of the DDQN algorithm is shown in Fig. 2.

As shown in Fig. 2, the structure of DDQN not only includes two Q-network for eliminating over estimation of action Q-value, but also includes an replay memory pool. Experience replay is one of the key technologies of DDQN. Each time the Agent executes a decision, it will get an experience tuple $e_t, e_t = \{s_t, a_t, r, s_{t+1}\}$, and the experience tuple e_t is stored in the replay memory pool $D_n, D_n = \{e_1, \dots, e_t, \dots, e_n\}$. Then, samples from the replay memory pool are randomly replayed for training instead of using continuous samples for training, as in Q-learning. In this way, the learning speed is accelerated, and the problems of correlated data and nonstationary distributions are alleviated.

This paper applies the DDQN algorithm to solve the task scheduling problem in the cloud computing environment, and we propose this novel scheduling algorithm as DDQN-TS. The algorithm has RL characteristics for solving sequential decision problems with satisfactory results, together with experience replay techniques that can effectively resolve data correlation and nonstationary distribution issues. We divide the entire scheduling process of DDQN-TS into two phases: the online decision phase and offline scheduling phase. The scheduling process of the DDQN-TS algorithm is shown in Algorithm 1.

Algorithm 1. DDQN-TS Algorithm

Input: User submits all tasks in real time.
Output: The task average response time.
Initialize replay memory pool capacity C , greedy coefficient ε , learning rate α , discount factor γ , minibatch Δ , and target Q-network parameter copy frequency N
Initialize Q-network parameters θ Initialize target $\hat{Q}$ -network parameters $\theta^- = \theta$
for each new task i that arrives **do** according to probability ε , randomly choose a VM as the action a
otherwise, according to the equation $a = \arg\max_a Q(s_t, a; \theta_t)$, select a VM as the action a
if task i can execute successfully on the VM, **then**
schedule task i according to the selected VM, obtain environmental reward value r , and transition from the current state s_t to the next decision time $t + 1$ state s_{t+1}
end if
Obtain the response time of task i
Store experience tuple (s_t, a, r, s_{t+1}) in replay memory pool D
for randomly extract minibatch experience tuples (s_t, a, r, s_{t+1}) from D **do**
 $y_t = r + \gamma \hat{Q}(s_{t+1}, \arg\max_a Q(s_{t+1}, a; \theta_t); \theta_t^-)$
perform a gradient descent step on $(y_t - Q(s_t, a; \theta_t))^2$, and update Q-network parameters θ according to the results
every N steps, reset $\hat{Q} = Q$
end for
Until the last task to execute is completed;
end for

4.4. DDQN-TS design principle

4.4.1. Two phase description

A good predictive model requires the neural networks to be trained on a large number of training sets. There are also some differences in the prediction models obtained by different training methods. Generally, two different training methods exist: online training and offline training. The online training strategy refers to training, adjusting network parameters, and making decisions simultaneously. This method has the advantage of being able to adjust the network parameters in real time according to changes in the external environment, and it is applicable to scenarios that require real-time changes to the environment, for example, large online game scenarios. The offline training strategy refers to fully training the neural network with a large number of generated training sets to obtain a network prediction model with specified parameters. Then, the obtained prediction model is used for prediction and testing. The advantage of this method is that after sufficient training, the prediction accuracy of the model can be improved. Since the environment of the application scenario does not need to be changed frequently and we aim to achieve high prediction accuracy and predictive integrity, we use a combination of online decision making and offline training to achieve intelligent scheduling.

According to Algorithm 1, we can see how the DDQN algorithm is applied to solve the task scheduling problem. First, the purpose of the online decision-making phase is to assign a suitable VM to each task that arrives in real time. Taking into account the contin-

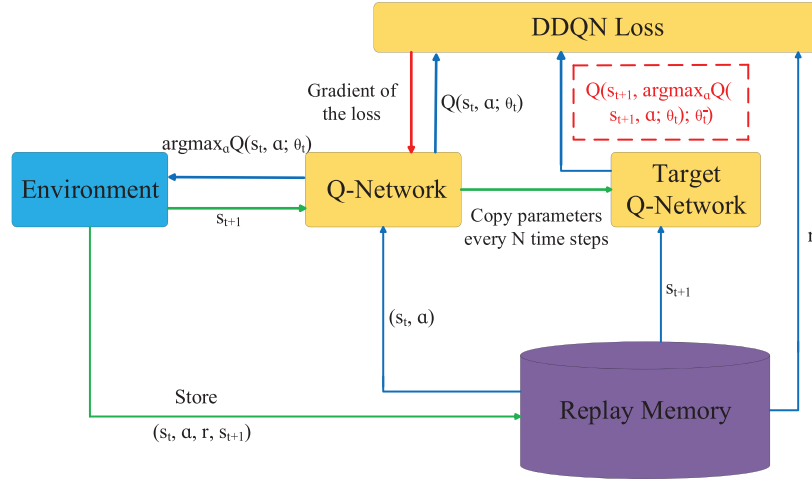


Fig. 2. The architecture of DDQN.

uous decision-making ability of Q-learning, the Q-learning technique is adopted in the online decision-making phase. Each time a new task arrives, the DDQN agent observes the current state s_t of the environment and then uses the Q-network to estimate the Q-values $Q(s_t, a; \theta_t)$ of all available VM instances in the current state. Next, a VM is assigned to the task based on a specific policy. After executing the task, the DDQN agent immediately receives a reward value r based on the selected action and transitions to the next state s_{t+1} . At the same time, the agent obtains the response time of the task.

Next, whenever performing a new task, that is, after making an online intelligent decision, an experience tuple e_t is generated and stored in the replay memory pool. Once the training begins, the DDQN agent randomly selects the experience tuple e_t from the replay memory pool for training. Therefore, agents can learn not only from current experience but also from previous experience. In this way, a more accurate action value function can be obtained. Through continuous training, the parameter θ of the Q-network is continuously updated according to the difference between the predicted action Q-value and the real action Q-value. When the difference between the two values becomes stable close to 0, the Q-network is fully trained, and every subsequent online decision can assign the most appropriate virtual machine to a newly arrived task. That is, each task can be assigned to obtain the minimum task response time after execution is completed.

4.4.2. Action space and state space

In our model, the number of VMs leased is limited. Each VM instance has a sufficiently long task waiting queue, and once the tasks enter the waiting queue, they can be executed. Therefore, the action space can be represented as the collection of all VM instances $A = [VM_1, VM_2, \dots, VM_m]$, where m is the total number of rented VM instances.

Every time a new task i arrives, we assume that the scenes state at this time is S_t^i . In our model, we use the state of the VM to simulate the state space; therefore, the state vector at this time can be expressed as $S_t^i = [VM_{1,i}^{idle}, VM_{2,i}^{idle}, \dots, VM_{m,i}^{idle}]$, where $VM_{m,i}^{idle}$ represents the earliest idle time of each VM after task i is executed. Once the task scheduler performs a specific action a for task i , it will transition from task i to the next task $i+1$. Additionally, the state S_t^i is converted to S_{t+1}^{i+1} , where $S_{t+1}^{i+1} = [VM_{1,i+1}^{idle}, VM_{2,i+1}^{idle}, \dots, VM_{m,i+1}^{idle}]$.

4.4.3. Reward value design

The reward value is used to evaluate the merits and demerits of each scheduling process's selected actions. A well-trained neural network should have the ability to evaluate positive situations so that the agent can reasonably allocate the limited resources according to the QoS request of users and thus achieve efficient task scheduling. In our model, the reward value can be expressed as Eq. (16).

$$rewards = \begin{cases} 1 - \frac{VM_{ij}^{idle}}{\max\{VM_k^{idle}\}}, & \text{if task } i \text{ met inequality} \\ -1, & \text{else} \end{cases} \quad \text{equation (7)} \quad (16)$$

where $\max\{VM_k^{idle}\}$ is the time after task i is assigned to execute on VM k such that the idle time of VM k is higher than that generated after task i is executed on other VMs. Suppose that there are four VMs with execution speeds of 200, 800, 1500, and 2000. The arrival time of task i waiting for execution is 0.8, and the size of task i is 1000. The reward value when task i is assigned to each VM for execution is shown in Table 2.

Table 2 shows that the maximum reward value is obtained when task i is assigned to VM_4 at this time, which means the most suitable VM for executing task i is VM_4 . When the arrival time of task i is certain, task i is assigned to execute on VM j so that VM j has the shortest idle time before executing the next task. Then, scheduling task i to VM j to perform this action results in the maximum reward. Clearly, when performing tasks, our aim is to obtain the maximum total reward that minimizes the total response time of the task to achieve the minimal average response time.

5. Experiments and analysis

In this section, we assess the performance of our proposed DDQN-TS method through a series of simulation experiments. First, we introduce the experimental environment, the structure of the Q-network, the hyperparameters, and the comparison algorithms used in the experiments. Then, we describe four sets of comparative experiments (experiments with the Random, Google, and Alibaba benchmarks) and analyze the experimental results.

Table 2

Example of reward value acquisition.

	VM ₁	VM ₂	VM ₃	VM ₄
The earliest idle time	1.0	0.7	1.8	0.8
The arrival time of task <i>i</i>	0.8	0.8	0.8	0.8
The execution start time	1.0	0.8	1.8	0.8
Whether the SLA is satisfied	no	yes	yes	yes
The execution time	0	1.25	0.67	0.5
The completion time	1.0	2.05	2.47	1.3
The earliest idle time before executing the next task	1.0	2.05	2.47	1.3
Reward	−1.0	0.66	0.59	0.78

5.1. Experimental setup

5.1.1. Experimental running environment

In this paper, all experiments are run on a Windows 10 operating system employing the Python 3.6 environment with TensorFlow 1.13, and the corresponding hardware is a Lenovo Pro with 8 GB RAM and a 2.4 GHz Intel Core i5 processor.

5.1.2. Q-network structure and hyperparameters

In the framework of our proposed task scheduling algorithm, we use a simple backpropagation (BP) neural network to construct the Q-network and the target Q-network. Since the two networks are identical, the two networks contain two hidden layers of 50 neurons. Thus, with the input and output layers, the neural network contains a total of 4 layers. The BP neural network is a multilayer feedforward neural network trained according to the error backpropagation algorithm, which includes forward propagation and error backpropagation of signals. In forward propagation, the input signal acts on the output node through the hidden layer to produce an output signal. If the actual output does not match the expected output, the error is backpropagated to reduce the difference between the actual output and the expected output. The error backpropagation adjusts the output weight of the input layer by layer through the hidden layer to adjust each the neurons' weights in each layer. After repeated learning and training to determine the network parameters that lead to the smallest error, the error becomes stable, and the training is terminated. The parameter settings used for the experiment are listed in Table 3.

5.1.3. Three classic algorithms for comparison

To evaluate the performance of our proposed DDQN-TS algorithm, we compare it with three classic algorithms: the random algorithm, round-robin algorithm, and DQN algorithm.

The random task scheduling method (Random) is relatively simple. Whenever a task arrives, a virtual machine is randomly assigned to the task. The principle of the round-robin task scheduling method (RR) is to assign real-time incoming tasks to all the virtual machines in turn. After all the virtual machines are allocated, the next round of loops is performed. The DQN algorithm is cur-

rently a relatively popular machine learning algorithm that has made significant breakthroughs in the AI field and is widely used in in-game scenarios. Because the DQN algorithm is also an unsupervised learning algorithm, it can self-learn an optimal allocation strategy based on many tasks. Therefore, the DQN algorithm has been applied to solve the task scheduling problem in the cloud environment and has achieved excellent results.

5.2. Experiments with different benchmarks

To test the performance of our proposed task scheduling method, we first train the model and then test the model. The training process requires a large number of task sets, so we randomly generate 50,000 independent tasks to fully train the model. We consider that both the Google workload traces [45] and Alibaba workload traces [46] are equally sampled and measured. Here, we assume that each task arrives at the task waiting queue with a time interval of 0.5 s. Using the same resources, we fully train the models of the DQN and DDQN algorithms. As shown in Fig. 3, both algorithms converge, but the DDQN algorithm converges when it is trained to the 24000th task, while the DQN algorithm converges when it reaches the 32000th task. The convergence results show that our proposed DDQN-TS is comparable to the DQN scheduling method. Our proposed scheduling method effectively alleviates the overestimation of the action Q-value, thereby accelerating the convergence of the network.

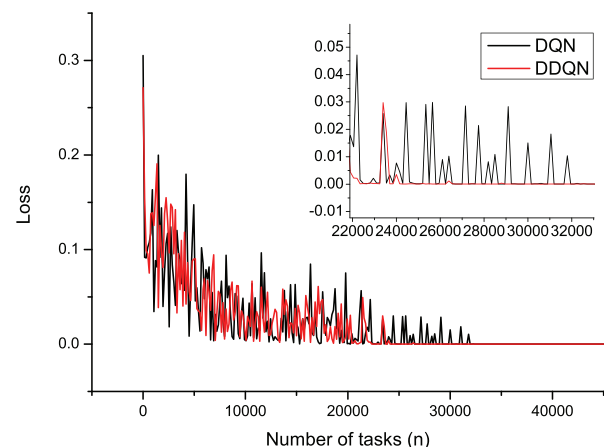
5.2.1. Experiments with the Random Benchmark

We select 1000, 2000, 4000, and 8000 tasks from the 50,000 randomly generated independent tasks for testing. Fig. 4 shows the task completion rate of all scheduling methods for different numbers of tasks. Our proposed task scheduling method DDQN-TS has the best task completion rate. Fig. 5 shows the average response

Table 3

Experimental parameter settings.

Parameters	Value
Number of VMs	4
VM execution speed	[200, 800, 1500, 2000]
Greedy coefficient ε	0.5
Discount factor γ	0.9
Learning rate α	0.1
Minibatch Δ	32
Replay memory pool capacity C	10000
Target Q-network parameter copy frequency N	20
Activation function	ReLU
Optimizer	Gradient descent optimizer
Loss function	Mean square error

**Fig. 3.** Comparison of convergence performance.

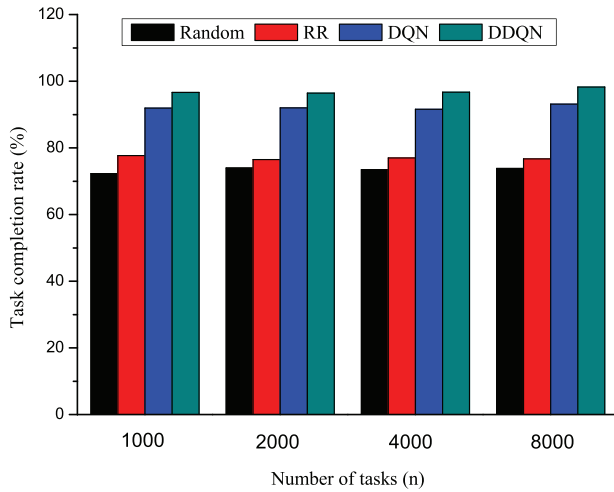


Fig. 4. Task completion rate of four algorithms using the Random Benchmark.

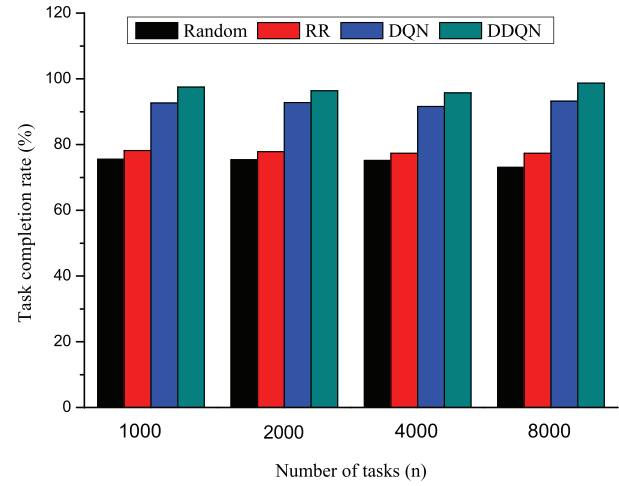


Fig. 6. Task completion rate of four algorithms using the Google Benchmark.

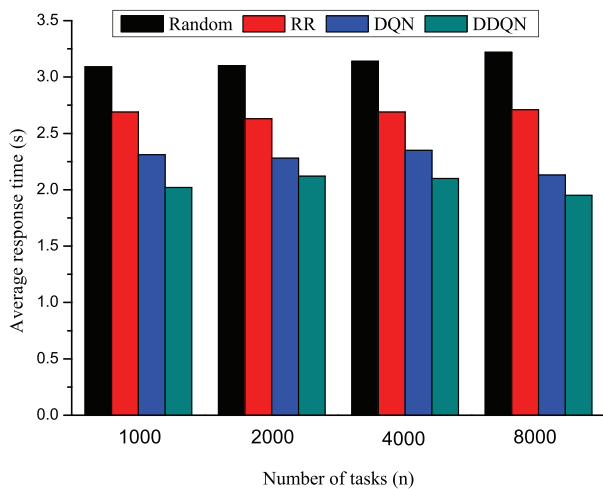


Fig. 5. Average response time of four algorithms using the Random Benchmark.

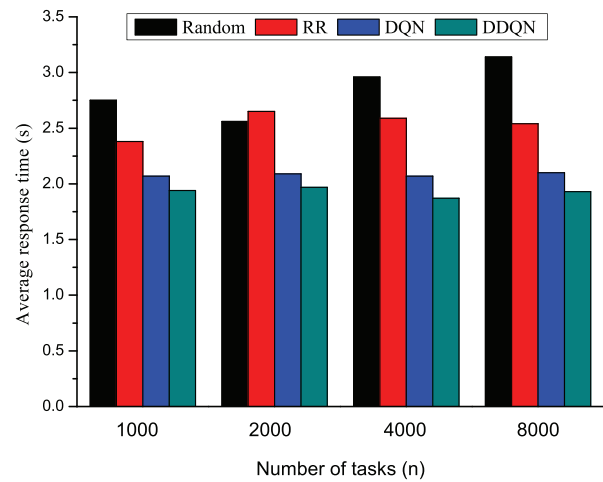


Fig. 7. Average response time of four algorithms using the Google Benchmark.

time of all tasks after the execution of the task: DDQN-TS has the shortest average response time.

5.2.2. Experiments with the Google Benchmark

To verify the universality of our proposed task scheduling method, we sort the Google Benchmark and extract some (1000, 2000, 4000, and 8000 tasks) of a large number of independent tasks for testing. As shown in Fig. 6, our proposed task scheduling method DDQN-TS has the best task completion rate. As shown in Fig. 7, DDQN-TS also has the shortest average response time.

5.2.3. Experiments with the Alibaba Benchmark

To verify our proposed task scheduling method's universality, we also test the algorithm with the real Alibaba Benchmark. As shown in Fig. 8 and Fig. 9, our proposed task scheduling method DDQN-TS has apparent advantages in terms of the task completion rate and average response time.

5.2.4. Extended experiments

To verify whether our proposed task scheduling method DDQN-TS is scalable and whether it is suitable for a more complex state space, we increase the number of VMs to 8 and set the speed of the VMs to [200, 600, 800, 1000, 1200, 1500, 1600, 2000]. Similarly,

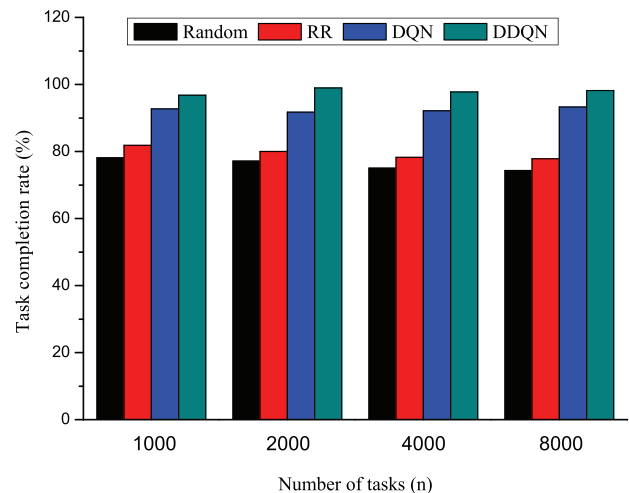


Fig. 8. Task completion rate of four algorithms using the Alibaba Benchmark.

we conduct a comparative experiment on the convergence performance of the algorithm. Except for the number of VMs and the execution speeds, which are different from those in the above

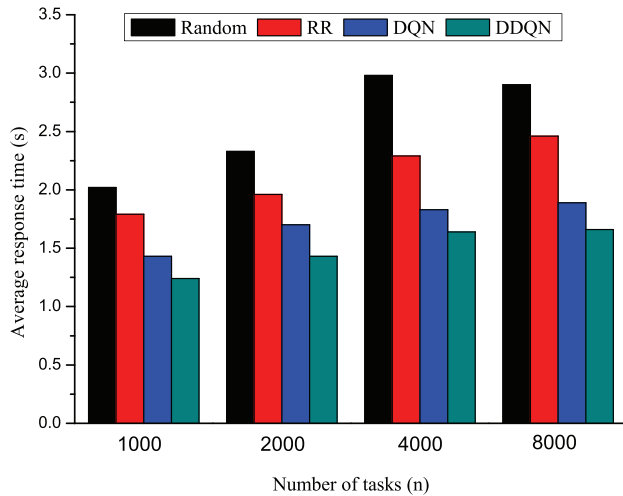


Fig. 9. Average response time of four algorithms using the Alibaba Benchmark.

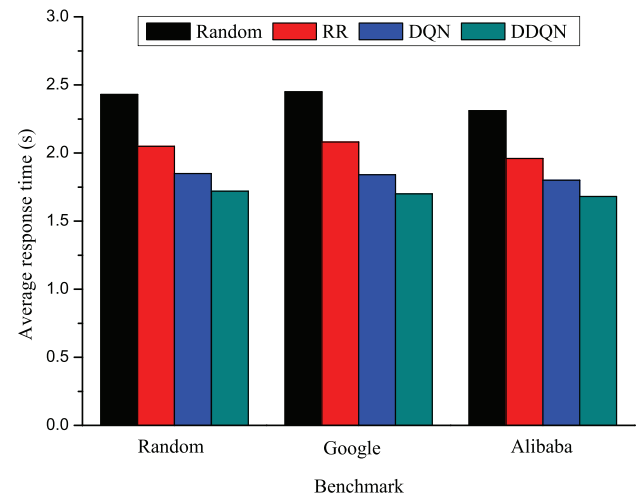


Fig. 12. Average response time of four algorithms with different benchmarks.

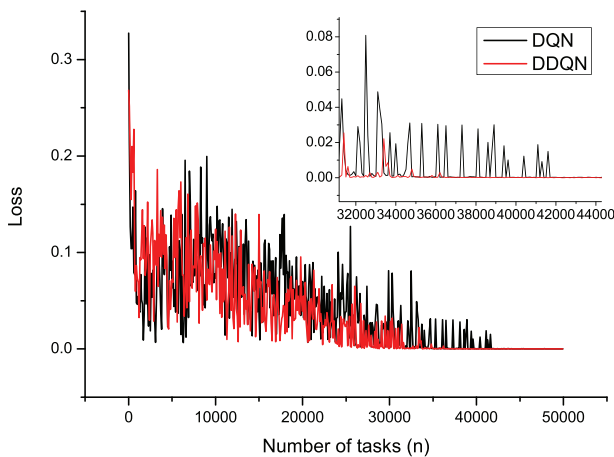


Fig. 10. Convergence performance with different algorithms.

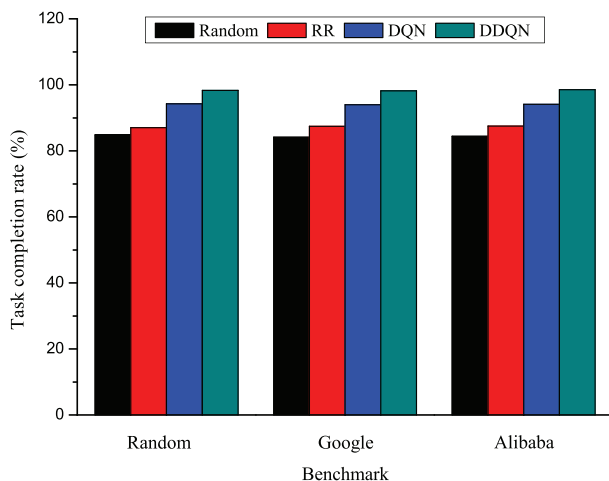


Fig. 11. Task completion rate of four algorithms with different benchmarks.

experiment, all the experimental parameters and resources are similar to those with the 4 VMs. As shown in Fig. 10, the DDQN algorithm converges at the 33500th task, and the DQN algorithm

converges at the 42000th task. As the state space becomes more complex, the convergence speed generally slows, but the DDQN algorithm converges much earlier than the DQN algorithm.

To reduce the repetitiveness of the experiment, next, we only test the performance of the algorithm with 8000 tasks on all three benchmarks. As shown in Figs. 11 and 12, our proposed task scheduling method DDQN-TS has apparent advantages in terms of task completion rate and average response time.

Combining all the experimental results, we find that our proposed task scheduling method DDQN-TS effectively alleviates the overestimation of the action Q-value by the DRL and DQN scheduling methods, thereby accelerating the convergence of the algorithm. Moreover, compared to the three classic algorithms, DDQN-TS has apparent advantages in terms of the task completion rate and average response time. The results of multiple comparison experiments also show that DDQN-TS has good adaptability, robustness and scalability.

6. Conclusions and future work

To achieve efficient real-time task scheduling in the cloud environment, we propose a novel task scheduling method based on the DDQN algorithm under the constraint of QoS. According to the experimental results, our proposed task scheduling method DDQN-TS effectively alleviates the overestimation of the action Q-value during the scheduling process and accelerates the convergence of the algorithm. Compared to other classic algorithms, our proposed method achieves the lowest average task response time while ensuring a high task completion rate. In general, our proposed task scheduling method DDQN-TS outperforms the other methods. In future work, we intend to apply our proposed method to a more complex cloud environment with proper consideration of energy consumption and economic benefits.

CRedit authorship contribution statement

Zhao Tong: Supervision, Writing-review & editing, Methodology. **Feng Ye:** Writing -original draft, Data curation. **Bilan Liu:** Data curation. **Jinhui Cai:** Formal analysis. **Jing Mei:** Writing -review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Program of the National Natural Science Foundation of China (Grant Nos. 62072174, 61502165, 61602170), the National Natural Science Foundation of Hunan Province, China (Grant No. 2020JJ5370), and the National Outstanding Youth Science Program of the National Natural Science Foundation of China (Grant No. 61625202).

References

- [1] A. Fox, R. Griffith, A. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, et al., Above the clouds: A Berkeley view of cloud computing, Dept. Electrical Eng. and Comput. Sciences, University of California, Berkeley, Rep. UCB/EECS 28 (13) (2009) 2009.
- [2] A. Celesti, M. Fazio, A. Galletta, L. Carnevale, J. Wan, M. Villari, An approach for the secure management of hybrid cloud-edge environments, *Future Generation Computer Systems* 90 (2019) 1–19.
- [3] G. Mei, N. Xu, J. Qin, B. Wang, P. Qi, A survey of internet of things (iot) for geohazard prevention: Applications, technologies, and challenges, *IEEE Internet of Things Journal* 7 (5) (2019) 4371–4386.
- [4] M. Shahradd, J. Balkind, D. Wentzlaff, Architectural implications of function-as-a-service computing, in: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, 2019, pp. 1063–1075.
- [5] D. Cui, Z. Peng, J. Xiong, B. Xu, W. Lin, A reinforcement learning-based mixed job scheduler scheme for grid or iaas cloud, *IEEE Annals of the History of Computing* 8 (4) (2020) 1030–1039.
- [6] T. Zhao, D. Xiaomei, C. Hongjian, M. Jing, Ddmts: A novel dynamic load balancing scheduling scheme under sla constraints in cloud computing, *Journal of Parallel and Distributed Computing* 149 (2021) 138–148.
- [7] M. Kumar, S.C. Sharma, A. Goel, S.P. Singh, A comprehensive survey for scheduling techniques in cloud computing, *Journal of Network and Computer Applications* 143 (2019) 1–33.
- [8] Z. Tong, H. Chen, X. Deng, K. Li, K. Li, A novel task scheduling scheme in a cloud computing environment using hybrid biogeography-based optimization, *Soft Computing* 23 (21) (2019) 11035–11054.
- [9] M. Ghobaei-Arani, S. Jabbehdari, M.A. Pourmina, An autonomic resource provisioning approach for service-based cloud applications: A hybrid approach, *Future Generation Computer Systems* 78 (2018) 191–210.
- [10] X. Chen, W. Ni, I.B. Collings, X. Wang, S. Xu, Automated function placement and online optimization of network functions virtualization, *IEEE Transactions on Communications* 67 (2) (2018) 1225–1237.
- [11] L. Bao, C. Wu, X. Bu, N. Ren, M. Shen, Performance modeling and workflow scheduling of microservice-based applications in clouds, *IEEE Transactions on Parallel and Distributed Systems* 30 (9) (2019) 2114–2129.
- [12] J. Bi, H. Yuan, M. Zhou, Temporal prediction of multiapplication consolidated workloads in distributed clouds, *IEEE Transactions on Automation Science and Engineering* 16 (4) (2019) 1763–1773.
- [13] N. Sharma, S. Maurya, Sla-based agile vm management in cloud & datacenter, in: 2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon), IEEE, 2019, pp. 252–257.
- [14] Z. Tang, X. Zhang, K. Li, K. Li, An intermediate data placement algorithm for load balancing in spark computing environment, *Future Generation Computer Systems* 78 (2018) 287–301.
- [15] A. Alelaiwi, Evaluating distributed iot databases for edge/cloud platforms using the analytic hierarchy process, *Journal of Parallel and Distributed Computing* 124 (2019) 41–46.
- [16] H. Xu, R. Li, C. Pan, K. Li, Minimizing energy consumption with reliability goal on heterogeneous embedded systems, *Journal of Parallel and Distributed Computing* 127 (2019) 44–57.
- [17] Z. Zhou, F. Li, H. Zhu, H. Xie, J.H. Abawajy, M.U. Chowdhury, An improved genetic algorithm using greedy strategy toward task scheduling optimization in cloud environments, *Neural Computing and Applications* 32 (6) (2020) 1531–1541.
- [18] Z. Tong, H. Chen, X. Deng, K. Li, K. Li, A scheduling scheme in the cloud computing environment using deep q-learning, *Information Sciences* 512 (2020) 1170–1191.
- [19] A. Tchernykh, U. Schwiigelsohn, E.-G. Talbi, M. Babenko, Towards understanding uncertainty in cloud computing with risks of confidentiality, integrity, and availability, *Journal of Computational Science* 36 (2019) 100581.
- [20] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fiedland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [21] H.-T. Joo, K.-J. Kim, Visualization of deep reinforcement learning using grad-cam: How ai plays atari games?, in: 2019 IEEE Conference on Games (CoG), IEEE, 2019, pp. 1–2.
- [22] Y. Wang, H. Liu, W. Zheng, Y. Xia, Y. Li, P. Chen, K. Guo, H. Xie, Multi-objective workflow scheduling with deep-q-network-based multi-agent reinforcement learning, *IEEE Access* 7 (2019) 39974–39982.
- [23] L. Liu, J. Hodgins, Learning to schedule control fragments for physics-based characters using deep q-learning, *ACM Transactions on Graphics (TOG)* 36 (3) (2017) 1–14.
- [24] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30, 2016, pp. 2094–2100.
- [25] Z. Zhou, H. Xie, F. Li, A novel task scheduling algorithm integrated with priority and greedy strategy in cloud computing, *Journal of Intelligent & Fuzzy Systems* 37 (4) (2019) 4647–4655.
- [26] T. Zhao, D. Xiaomei, Y. Feng, B. Sunitha, X. Xueli, P. Yi, Adaptive computation offloading and resource allocation strategy in a mobile edge computing environment, *Information Sciences* 537 (2020) 116–131.
- [27] X. Liang, X. Du, G. Wang, Z. Han, A deep reinforcement learning network for traffic light cycle control, *IEEE Transactions on Vehicular Technology* 68 (2) (2019) 1243–1253.
- [28] Z. Tong, X. Deng, H. Chen, J. Mei, H. Liu, QI-heft: a novel machine learning scheduling scheme base on cloud computing environment, *Neural Computing and Applications* 32 (10) (2020) 5553–5570.
- [29] Q. Liu, X. Li, H. Liu, Z. Guo, Multi-objective metaheuristics for discrete optimization problems: A review of the state-of-the-art, *Applied Soft Computing* 93 (2020) 106382.
- [30] M. Aldossary, K. Djemame, I. Alzamil, A. Kostopoulos, A. Dimakis, E. Agiatzidou, Energy-aware cost prediction and pricing of virtual machines in cloud computing environments, *Future Generation Computer Systems* 93 (2019) 442–459.
- [31] L. Coppolino, S. D'Antonio, V. Formicola, G. Mazzeo, L. Romano, Vise: Combining intel sgx and homomorphic encryption for cloud industrial control systems, *IEEE Transactions on Computers* 70 (5) (2020) 711–724.
- [32] C. Tsiganos, S. Nastic, S. Dustdar, Towards resilient internet of things: Vision, challenges, and research roadmap, in: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), IEEE, 2019, pp. 1754–1764.
- [33] M. Liwang, J. Wang, Z. Gao, X. Du, M. Guizani, Game theory based opportunistic computation offloading in cloud-enabled iot, *IEEE Access* 7 (2019) 32551–32561.
- [34] Y. Liu, L.L. Njilla, J. Wang, H. Song, An lstm enabled dynamic stackelberg game theoretic method for resource allocation in the cloud, in: 2019 International Conference on Computing, Networking and Communications (ICNC), IEEE, 2019, pp. 797–801.
- [35] S. Wang, X. Li, R. Ruiz, Performance analysis for heterogeneous cloud servers using queueing theory, *IEEE Transactions on Computers* 69 (4) (2019) 563–576.
- [36] H.A. Balla, C.G. Sheng, J. Weipeng, Reliability enhancement in cloud computing via optimized job scheduling implementing reinforcement learning algorithm and queueing theory, in: 2018 1st International Conference on Data Intelligence and Security (ICDIS), IEEE, 2018, pp. 127–130.
- [37] B. Demirel, A. Ramaswamy, D.E. Quevedo, H. Karl, Deepcas: A deep reinforcement learning algorithm for control-aware scheduling, *IEEE Control Systems Letters* 2 (4) (2018) 737–742.
- [38] M. Cheng, J. Li, S. Nazarian, Drl-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers, in: 2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC), IEEE, 2018, pp. 129–134.
- [39] Z. Zhou, J. Abawajy, M. Chowdhury, Z. Hu, K. Li, H. Cheng, A.A. Alelaiwi, F. Li, Minimizing sla violation and power consumption in cloud data centers using adaptive energy-aware algorithms, *Future Generation Computer Systems* 86 (2018) 836–850.
- [40] Y. Wei, L. Pan, S. Liu, L. Wu, X. Meng, Drl-scheduling: An intelligent qos-aware job scheduling framework for applications in clouds, *IEEE Access* 6 (2018) 55112–55125.
- [41] P. Azad, N.J. Navimipour, An energy-aware task scheduling in the cloud computing using a hybrid cultural and ant colony optimization algorithm, *International Journal of Cloud Applications and Computing (IJCAC)* 7 (4) (2017) 20–40.
- [42] W. Dabney, Z. Kurth-Nelson, N. Uchida, C.K. Starkweather, D. Hassabis, R. Munos, M. Botvinick, A distributional code for value in dopamine-based reinforcement learning, *Nature* 577 (7792) (2020) 671–675.
- [43] M. El Chamie, D. Janak, B. Açıkmeşe, Markov decision processes with sequential sensor measurements, *Automatica* 103 (2019) 450–460.
- [44] R. Agarwal, D. Schuurmans, M. Norouzi, Striving for simplicity in off-policy deep reinforcement learning, *arXiv preprint arXiv:1907.04543*.
- [45] L. Ruan, X. Xu, L. Xiao, F. Yuan, Y. Li, D. Dai, A comparative study of large-scale cluster workload traces via multiview analysis, in: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), IEEE, 2019, pp. 397–404.
- [46] C. Lu, K. Ye, G. Xu, C.-Z. Xu, T. Bai, Imbalance in the cloud: An analysis on alibaba cluster trace, in: 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 2884–2892.



Parallel and Distributed Computing, etc. He is a member of the IEEE and CCF.

Zhao Tong received the Ph.D. in computer science from Hunan University, China, in 2014. He was a visiting scholar with the Georgia State University from 2017 to 2018. He is currently an Associate Professor in the College of Information Science and Engineering of Hunan Normal University. His research interests include modeling and scheduling for parallel and distributed computing systems. He has published more than 15 research papers in international conferences and journals such as the IEEE Transactions on Parallel and Distributed Systems, Information Sciences, Neural Computing and Applications, Soft Computing, Journal of



Jinhui Cai received the BE degree from Qilu University of Technology, Jinan, China in 2019. He is currently pursuing a MS degree in the College of Information Science and Engineering of Hunan Normal University, Changsha, China. His research interests include cloud-edge collaborative computing, objective optimization, machine learning, and artificial intelligence.



Feng Ye received the BE degree from Hunan Institute of Science and Technology, Yueyang, China in 2018. He is currently pursuing a MS degree in the College of Information Science and Engineering of Hunan Normal University, Changsha, China. His research interests include cloud computing, mobile edge computing, objective optimization, task scheduling, machine learning, and artificial intelligence. He has published 3 research articles in international conference and journals.



Jing Mei received the Ph.D. in computer science from Hunan University, China, in 2015. She is currently an assistant professor in the College of Information Science and Engineering at Hunan Normal University. Her research interests include parallel and distributed computing, cloud computing, etc. She has published 12 research articles in international conference and journals, such as IEEE Transactions on Computers, IEEE Transactions on Service Computing, Cluster Computing, Journal of Grid Computing, Journal of Supercomputing.



Bilan Liu received the BE degree from Hunan Institute of Science and Technology, Yueyang, China in 2020. She is currently pursuing a MS degree in the College of Information Science and Engineering of Hunan Normal University, Changsha, China. Her research interests include Cloud-edge collaborative computing, deep learning algorithms and combinatorial optimization.